

COMPUTAÇÃO EM NUVEM 2026.1

INFRAESTRUTURA como CÓDIGO

EQUIPE:

DAVI SOUSA FACUNDO – 565910
EDIVAR CRUZ FILHO – 565857
MARCOS EMANUEL DIAS – 568099
ANNA BEATRYZ FERREIRA – 566366
RANDSON DA SILVA ALVES – 568002

SUMÁRIO

FUNDAMENTOS, ABORDAGENS E IDEMPOTÊNCIA

PROVISIONAMENTO DE INFRAESTRUTURA

GERENCIAMENTO DE CONFIGURAÇÃO

ECOSSISTEMA ESPECÍFICO DA AWS

CICLO DE VIDA, GITOPS E CI/CD

O QUE É

IAC

Substituição Manual

A Infraestrutura como Código (IaC) é a prática de substituir processos manuais e cliques em painéis web pela escrita de arquivos de configuração declarativos e versionáveis.

Cultura de Software

Ao transformar servidores, redes e bancos de dados em código de verdade, trazemos a agilidade, segurança e os testes da engenharia de software diretamente para a nossa infraestrutura.



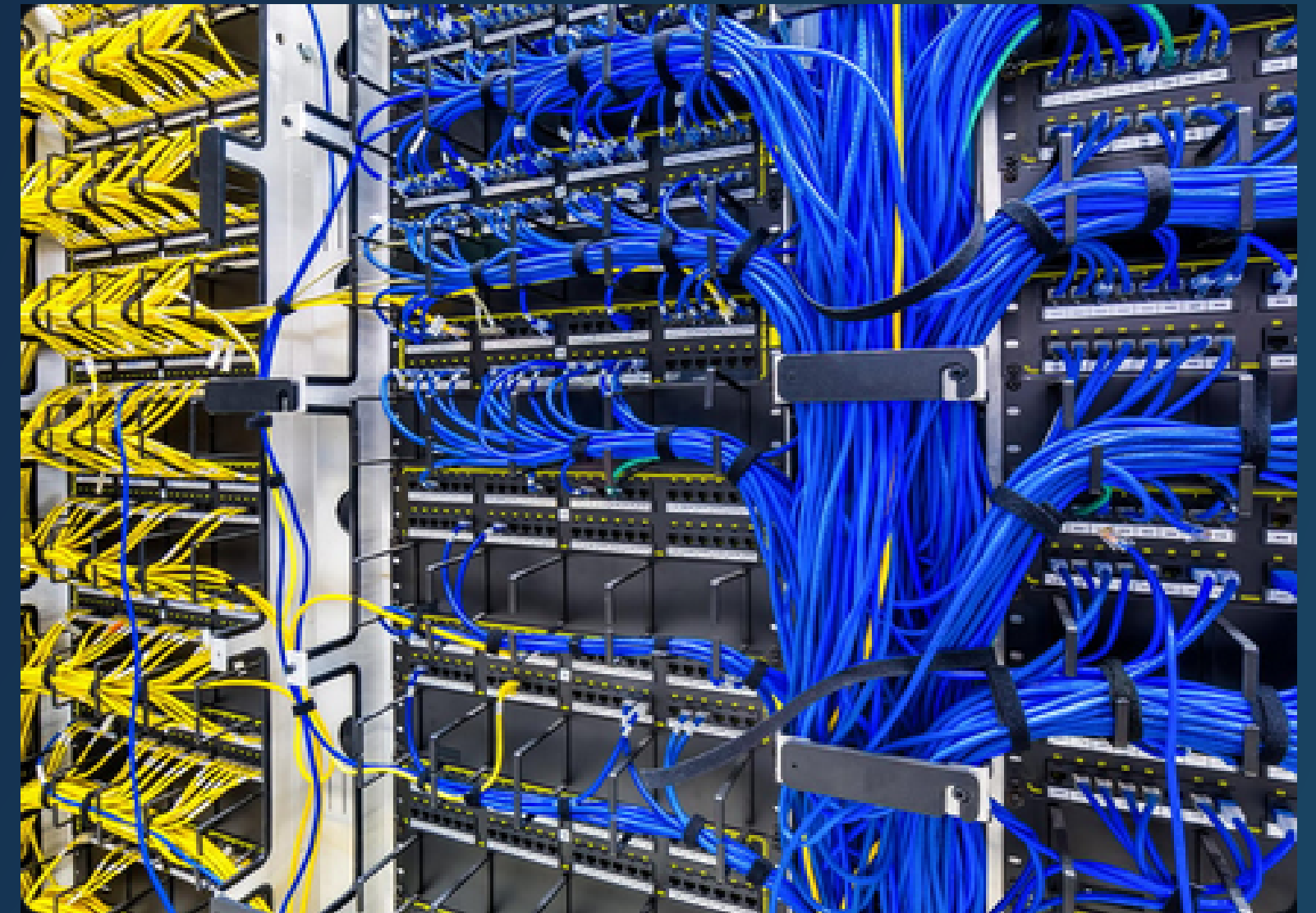
O VILÃO: CONFIGURATION DRIFT

O que é o "Drift"?

É a divergência silenciosa que ocorre quando ambientes que deveriam ser idênticos (como homologação e produção) começam a se diferenciar devido a intervenções e ajustes manuais diretos no console da nuvem.

Causa: Configurações rápidas "ad-hoc" sem registro.

Solução: Terraform como única fonte de verdade.



DECLARATIVO VS IMPERATIVO

✓ Abordagem Declarativa

Você define o **estado final desejado**. O Terraform analisa a infraestrutura atual e faz todo o cálculo necessário para atingir o objetivo sem que você escreva o passo a passo.

Analogia: Inserir o endereço final de destino em um GPS.

➡ Abordagem Imperativa

Focada nas instruções passo a passo. Os scripts convencionais exigem que você programe cada comando de criação de rede, segurança e servidores, correndo riscos de quebra caso um passo falhe.

Analogia: Seguir orientações de curva a curva escritas à mão.



EXEMPLO PRÁTICO NA NUVEM

O Banco de Dados de 20GB

Se você rodar o Terraform solicitando a criação de um banco de dados de 20GB:

Na primeira execução: O banco de dados é criado do zero

Nas execuções seguintes: O Terraform apenas valida que o banco de 20GB já existe e não faz mais nada. Nenhum banco duplicado será criado.



PROVISIONAMENTO DE INFRAESTRUTURA

FOCO NA "CASCA BRUTA" DO AMBIENTE

O que é provisionar?

É a etapa inicial da jornada de IaC, que compete a:

- Criação, alteração e destruição dos recursos de hardware virtual.
- Erguer a fundação do ambiente antes de qualquer configuração interna.

Elementos criados nesta etapa:

Redes (VPC)

Sub-redes e roteamento

Máquina (EC2)

Instâncias de computação

Bancos de dados (RDS)

Armazenamento e motores SQL/NoSQL

O PODER DAS FERRAMENTAS AGNÓSTICAS

COMUNICAÇÃO VIA API E FLEXIBILIDADE DE PROVEDOR

O que significa ser Agnóstico?

Capacidade de se comunicar via API com múltiplos provedores de nuvem (AWS, Azure, GCP) utilizando a mesma linguagem e ferramenta.

Como funciona por baixo dos panos?

A ferramenta traduz o seu arquivo de definição em chamadas de API específicas para o provedor de nuvem escolhido.

O Grande Alvo Estratégico:

Evitar o Vendor Lock-in (dependência comercial e técnica de um único fornecedor) e viabilizar arquiteturas híbridas ou Multicloud.

OS PRINCIPAIS PLAYERS DO MERCADO

PADRÃO DA INDÚSTRIA, OPEN-SOURCE E EVOLUÇÃO

TERRAFORM

O padrão absoluto do mercado. Utiliza a linguagem declarativa HCL.

OPENTOFU

A alternativa estritamente open-source, mantida pela Linux Foundation (derivada como um fork do Terraform).

PULUMI

Permite provisionar a casca da infraestrutura utilizando linguagens de programação tradicionais (como Python, TypeScript ou Go).

DESAFIOS NO MUNDO REAL

CONCORRÊNCIA E PERMISSÕES

Dificuldades Práticas no Provisionamento Agnóstico:

- Conflitos de Estado (State File): Sincronizar o arquivo que mapeia a nuvem real quando múltiplas pessoas alteram o código ao mesmo tempo. (Solução: Trava de estado/State Lock remoto).
- Ajuste de Permissões (IAM): Liberar credenciais de API robustas o suficiente para criar redes e bancos, mantendo o privilégio mínimo.

Limite de Atuação:

Garante a existência do recurso e a conectividade, sem gerenciar os arquivos ou softwares instalados dentro do sistema operacional.

GERENCIAMENTO DE CONFIGURAÇÃO

Objetivo

Padronizar, controlar e automatizar a configuração de servidores, recursos e garantir que todos os ambientes sejam iguais e reproduzíveis.

Definição da configuração
arquivo com o que o servidor deve ter

Ferramenta executa
Ela lê e aplica a configuração no servidor

Verificação contínua
Corrige caso mudanças aconteçam

Ferramentas: Ansible, Puppet



EXEMPLO PRÁTICO: CONFIGURANDO UM SERVIDOR WEB

Sem Gerenciamento

- Acessar o servidor manualmente via SSH
- Instalar pacotes digitando: `apt install nginx`
- Configurar arquivos tudo na mão

Demorado

Sujeito a erros

Difícil repetir

Com Ansible

Escrevemos o estado desejado em YAML:

```
- hosts: servidores tasks: - name:
  Instalar Nginx apt: name: nginx state:
  present - name: Iniciar serviço
  service: name: nginx state: started
```

Resultado:

`ansible-playbook setup.yml`

Um único comando e o servidor já sai com Nginx **instalado e rodando** automaticamente.

Em segundos, de forma totalmente padronizada e repetível.

CICLO DE VIDA GITOPS E CI/CD

**INFRAESTRUTURA
TRADUZIDA COMO CÓDIGO**



GIT

Com o código passando a viver no Git, exatamente como um código de aplicação comum, podemos introduzir novas práticas como: controle de versão, code review, e integração contínua na governança dos servidores. O Git passa a ser a fonte única da verdade para a arquitetura da empresa.

CICLO DE VIDA GITOPS E CI/CD

**INFRAESTRUTURA
TRADUZIDA COMO CÓDIGO**



GIT

É possível criar pipelines de CI/CD que sobem uma infraestrutura temporária completa, executam suítes de testes automatizados e garantem a qualidade de toda a aplicação em um ambiente isolado antes de destruir os recursos e autorizar o envio para a produção.

SOLUÇÕES ESPECÍFICAS DA AWS

AWS CDK

Permite a modelagem da IaC em linguagens de programação conhecidas, como Python ou JavaScript. Compila o código para o CloudFormation, para então provisionar a IaC

AWS CLOUDFORMATION

Provisiona IaC com arquivos JSON ou YAML. Tudo de forma automatizada

TERRAFORM

Ferramenta para configurar IaC pelo AWS CLI. Código próprio **.tf**,



IAC

Home

About

Content

Others

CRIANDO UMA INSTÂNCIA EC2 COM TERRAFORM



PASSO-A-PASSO

1. INSTALAR TERRAFORM CLI

2. INSTALAR E CONFIGURAR
AWS CLI

3. CONFIGURAR ARQUIVOS
TERRAFORM

4. CONFIGURANDO A
MÁQUINA EC2

5. INICIANDO A INSTÂNCIA

6. ENCERRANDO INSTÂNCIA
E AMBIENTE



INSTALANDO O TERRAFORM CLI

no Linux

1. Atualizar o sistema e instalar pacotes importantes

```
sudo apt-get update && sudo apt-get install -y gnupg software-properties-common
```

2. Instalando uma chave GPG da HashiCorp

```
wget -O- https://apt.releases.hashicorp.com/gpg | \  
gpg --dearmor | \  
sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg > /dev/null
```

3. Verificando a chave GPG

```
gpg --no-default-keyring \  
--keyring /usr/share/keyrings/hashicorp-archive-keyring.gpg \  
--fingerprint
```

3.1: A saída no terminal deve ser:

```
/usr/share/keyrings/hashicorp-archive-keyring.gpg  
-----  
pub   rsa4096 XXXX-XX-XX [SC]  
AAAA AAAA AAAA AAAA  
uid           [ unknown] HashiCorp Security (HashiCorp Package Signing) <security+packaging@hashicorp.com>  
sub   rsa4096 XXXX-XX-XX [E]
```

INSTALANDO O TERRAFORM CLI

no Linux

4. Adicione o repositório oficial da HashiCorp:

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/  
hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(grep -o  
P '(?<=UBUNTU_CODENAME=).*' /etc/os-release || lsb_release -cs) main" | sud  
o tee /etc/apt/sources.list.d/hashicorp.list
```

5. Passos finais

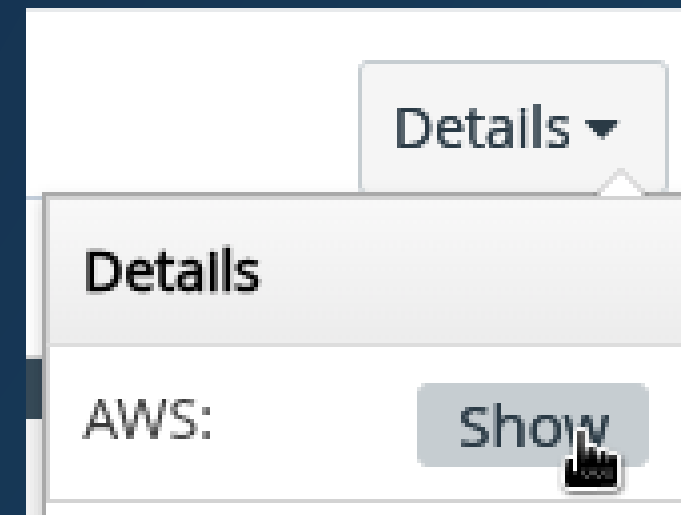
```
sudo apt update  
sudo apt-get install terraform
```


INSTALANDO O AWS CLI no Linux

1. Acesse o Sandbox da Instructure
2. Inicie o Lab



3. Após o carregamento do Lab, expanda **Details** e guarde **AccessKey** e **SecretKey**



INSTALANDO O AWS CLI no Linux

4. Abra o terminal da sua máquina e insira:

```
aws configure
```

5. Quando solicitado, insira AccessKey e SecretKey, confirme a região que deseja trabalhar e selecione a forma de saída (nossa escolhida foi JSON)

```
AWS Access Key ID [*****76U7]:  
AWS Secret Access Key [*****3oD0]:  
Default region name [us-east-1]:  
Default output format [json]:
```


CONFIGURANDO ARQUIVOS TERRAFORM

1. Crie uma pasta para sua IaC

```
mkdir aprendendo-iac-nuvem  
cd aprendendo-iac-nuvem
```

2. Abra uma IDE de sua preferência
(utilizamos VS Code)

```
code .
```

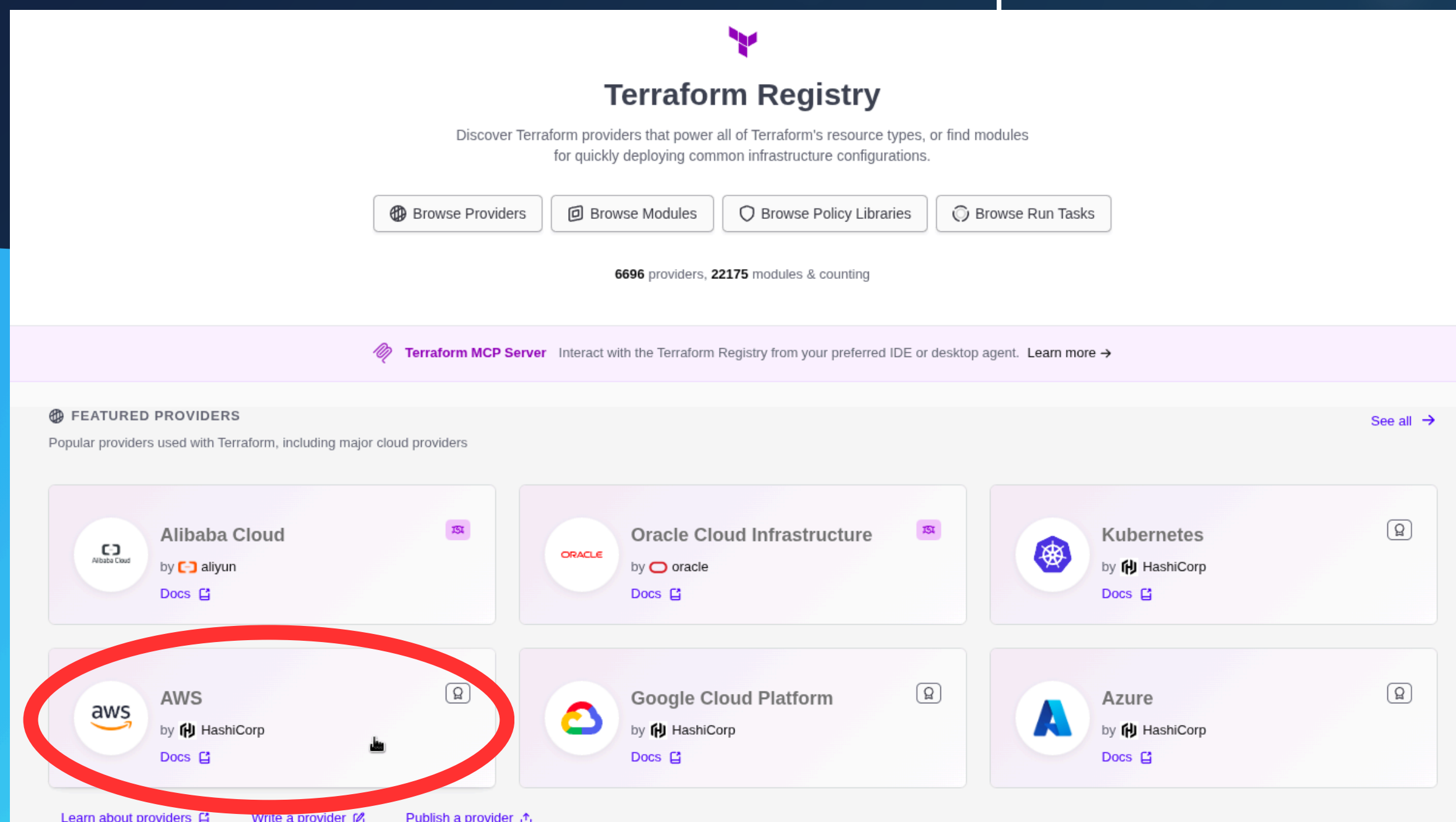
3. Crie um arquivo **terraform.tf**

CONFIGURANDO ARQUIVOS TERRAFORM

1. A HashiCorp oferece formas de se conectar ao provedor que você desejar, no site

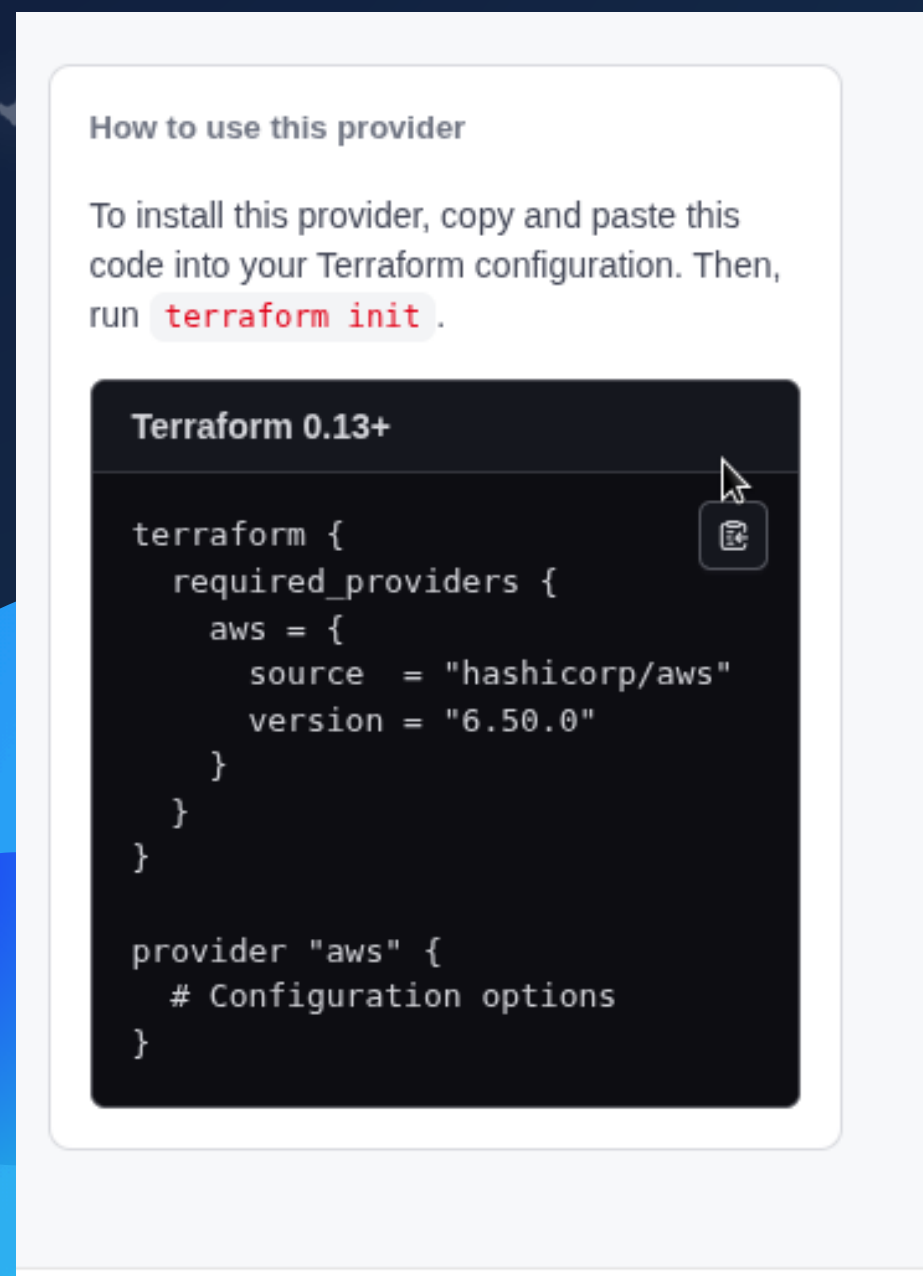
<https://registry.terraform.io>

2. Acesse o site e selecione o provedor AWS



CONFIGURANDO ARQUIVOS TERRAFORM

3. Na página do fornecedor, copie o código que aparece no canto inferior direito



4. Adicione ao arquivo terraform.tf:

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "6.50.0"
    }
  }
}
```

CONFIGURANDO MÁQUINA EC2

1. Crie um arquivo main.tf
2. Adicione o bloco de dados para buscar uma AMI:

Esta AMI vai ficar na variável `data.aws_ami.ubuntu.id`
(importante para o próximo passo)

```
data "aws_ami" "ubuntu" {  
  most_recent = true  
  
  filter {  
    name = "name"  
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]  
  }  
  
  owners = ["099720109477"] # Canonical
```

Blocos de dados (data) são colocados quando se quer buscar algo que existe fora do Terraform

Neste caso, ele procura pelo **Ubuntu 24.04 LTS**, versão **Server**, arquitetura de **64 bits** (amd64), tipo de virtualização **HVM** e armazenamento em **SSD** do tipo **gp3**.

Segurança: serve para verificar se o dono da imagem possui este ID (Canonical é a distribuidora oficial do Ubuntu)

CONFIGURANDO MÁQUINA EC2

3. No arquivo main.tf, adicione o bloco de recurso para a instância

```
resource "aws_instance" "app_server" {  
  ami          = data.aws_ami.ubuntu.id # preenchendo a AMI que buscamos no DATA  
  instance_type = "t2.micro"  
  
  root_block_device {  
    volume_type = "gp2"  
    volume_size = 8  
  }  
  
  tags = {  
    Name = "learn-terraform" O nome será esse  
  }  
}
```

OBS: o laboratório do Sandbox permite apenas usar volumes do tipo gp2, sendo assim, é necessário o bloco de código root_block_device para alterar o tipo do volume para gp2.

CONFIGURANDO MÁQUINA EC2

4. Inicialize o Terraform na sua pasta, através do terminal

```
terraform init
```

Aguarde a mensagem de sucesso

```
Initializing provider plugins found in the configuration...  
- Reusing previous version of hashicorp/aws from the dependency lock file  
- Using previously-installed hashicorp/aws v6.50.0  
  
Initializing the backend...  
  
Initializing provider plugins found in the state...  
- Reusing previous version of hashicorp/aws  
- Using previously-installed hashicorp/aws v6.50.0  
  
Terraform has been successfully initialized!  
  
You may now begin working with Terraform. Try running "terraform plan" to see  
any changes that are required for your infrastructure. All Terraform commands  
should now work.  
  
If you ever set or change modules or backend configuration for Terraform,  
rerun this command to reinitialize your working directory. If you forget, other  
commands will detect it and remind you to do so if necessary.
```

HORA DE EXECUTAR

1. No terminal:

```
terraform apply
```

Confirme com 'yes'
Aguarde a criação

2. Abra o EC2 no Console da AWS

3. Acesse as Instâncias

4. Busque pela instância com o nome que criamos no main.tf

Instâncias (1/2) Informações						
Conjuntos de filtros salvos Escolher conjunto de ... ▼		🔍 Localizar Instância por atributo ou tag (case-sensitive)				
☐	Name ✎ ▼	ID da instância	Estado da inst... ▼	Tipo de inst... ▼	Verificação de stat	Zona de dispon.
☑	learn-terraform	i-004b1f753f5503d6b	✅ Executando 🔍 🔍	t2.micro	🕒 Inicializando	us-east-1d

**DESCOBRINDO O STATUS DA
INFRAESTRUTURA**

```
terraform state list
```

**VERIFICANDO INFORMAÇÕES
DA INFRAESTRUTURA**

```
terraform show
```

ENCERRANDO A INSTÂNCIA

Comente o bloco de código que cria a instância

```
terraform apply
```

confirme 'yes'

Verifique no EC2 - Instâncias

☐	learn-terraform	i-004b1f753f5503d6b	Desligamento
--------------------------	-----------------	---------------------	--------------

ENCERRANDO O AMBIENTE

```
terraform destroy
```

confirme 'yes'



problema

Dificuldade de conciliar os requisitos de criação de uma máquina no laboratório da AWS, pois existem limitações por conta do preço.

O Terraform envia a requisição e retorna com erro, sendo impossibilitado de criar a máquina no EC2.

solução

Gerar um bloco de código dentro do resource que obrigue o Terraform a buscar um armazenamento do tipo que o laboratório permita.

```
root_block_device {  
  volume_type = "gp2"  
  volume_size = 8  
}
```


PERGUNTA:

Um analista de infraestrutura precisa garantir que um servidor de aplicação específico esteja rodando na nuvem. Ele executa o script do Terraform e o servidor é criado com sucesso. Se o analista executar o mesmo script idêntico logo em seguida, o princípio da idempotência garante que o Terraform não criará um segundo servidor duplicado, mas apenas validará que o estado atual da nuvem já corresponde ao desejado.

FINALIZAMOS POR AQUI
OBRIGADO

REFERÊNCIAS

- ATlassian. Infrastructure as Code: Declarative vs. Imperative automation. Disponível em: <https://www.atlassian.com/devops/infrastructure/infrastructure-as-code>. Acesso em: 14 jun. 2026.
- HASHICORP. Introduction to Infrastructure as Code (IaC) with Terraform. Disponível em: <https://developer.hashicorp.com/terraform/tutorials>. Acesso em: 14 jun. 2026.
- IBM. Introduction to DevOps. Curso online hospedado na plataforma Coursera. Acesso em: 14 jun. 2026.
- <https://developer.hashicorp.com/terraform/tutorials/aws-get-started/aws-create#write-configuration>
- <https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>
- <https://developer.hashicorp.com/terraform/tutorials/aws-get-started/install-cli>
- <https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-sign-in.html#cli-configure-sign-in-prerequisites>
- <https://aws.amazon.com/pt/cli/>
- <https://developer.hashicorp.com/terraform/tutorials/aws-get-started/infrastructure-as-code>
- <https://developer.hashicorp.com/terraform/tutorials/aws-get-started>
- <https://aws.amazon.com/pt/cloudformation/>
- <https://docs.aws.amazon.com/cdk/v2/guide/home.html>